

Universidad de San Carlos de Guatemala

Facultad de Ingeniería

Escuela de Ciencias y Sistemas

Estructura de Datos

Manual Técnico

Jennifer Yulissa Lourdes Taperio Manuel

Ingeniero: Luis Espino

Guatemala 24 de ago. De 24

Lista Circular Doble

La clase ListaCircularDoble maneja esta estructura. Cuando la lista se crea, comienza vacía. Al agregar una nueva publicación, si la lista estaba vacía, el nuevo nodo se convierte en el único nodo y se enlaza a sí mismo. Si ya hay nodos en la lista, el nuevo nodo se añade al final, ajustando los enlaces de los nodos para mantener la circularidad: el último nodo apunta al primero y viceversa.

El método mostrarPublicaciones recorre la lista desde el primer nodo y muestra la información de cada nodo, volviendo al inicio al final del recorrido, lo que destaca la naturaleza circular de la lista. El método navegarPublicaciones permite a los usuarios moverse entre publicaciones de manera interactiva, eligiendo si quieren ver el nodo anterior, el siguiente o salir del recorrido.

Para liberar memoria, el destructor y el método limpiar eliminan todos los nodos de la lista. Además, hay funciones para generar una representación gráfica de la lista en un archivo .dot, que luego se convierte en una imagen usando Graphviz

```
1 #include "ListaCircularDoble.h"
2 #include <iostream>
3 #include <fstream>
4 #include <string>
5
6 NodoCircular::NodoCircular(const std::string& correo, const std::string& contenido, const std::string& fecha, const std::string& hora)
7     : correo(correo), contenido(contenido), fecha(fecha), hora(hora), siguiente(nullptr), anterior(nullptr) {}
8
9 ListaCircularDoble::ListaCircularDoble() : cabeza(nullptr), cola(nullptr) {}
10
11 ListaCircularDoble::~ListaCircularDoble() {}
12
13 if (cabeza != nullptr) {
14     NodoCircular* actual = cabeza;
15     while (actual != nullptr) {
16         actual = actual->siguiente;
17     }
18 }
19
20 void ListaCircularDoble::agregarPublicacion(const std::string& correo, const std::string& contenido, const std::string& fecha, const std::string& hora) {
21     NodoCircular* nuevoNodo = new NodoCircular(correo, contenido, fecha, hora);
22     if (cabeza == nullptr) {
23         cabeza = cola = nuevoNodo;
24         nuevoNodo->siguiente = cabeza;
25         nuevoNodo->anterior = cabeza;
26     } else {
27         nuevoNodo->siguiente = cabeza;
28         nuevoNodo->anterior = cola;
29         cola->siguiente = nuevoNodo;
30         cabeza->anterior = nuevoNodo;
31         cola = nuevoNodo;
32     }
33 }
```

Lista Publicaciones

La clase `ListaDePublicaciones` se inicializa con una lista de usuarios (`listaUsuarios`) que permite verificar si un correo electrónico está registrado. El constructor inicializa los punteros cabeza y cola en `nullptr`, y el destructor asegura que todas las publicaciones sean eliminadas correctamente para evitar pérdidas de memoria.

Al agregar una nueva publicación con el método `agregarPublicacion`, primero se verifica si el correo del usuario existe en la lista de usuarios. También se asegura de que todos los campos de la publicación (correo, contenido, fecha y hora) estén completos. Si se cumplen estas condiciones, se crea una nueva publicación y se añade al final de la lista. Si la lista está vacía, el nuevo nodo se convierte en la cabeza y la cola; si no, se ajustan los enlaces de los nodos para insertar el nuevo nodo al final de la lista.

Para eliminar una publicación, el método `eliminarPublicacion` recorre la lista buscando una publicación que coincida con el correo, la fecha y la hora especificados. Si encuentra una coincidencia, ajusta los enlaces de los nodos adyacentes para remover el nodo de la lista y luego lo elimina. Si no encuentra la publicación, informa al usuario.

El método `mostrarPublicaciones` recorre toda la lista desde la cabeza, mostrando la información de cada publicación. Por otro lado, `mostrarTopPublicaciones` realiza un análisis más detallado: primero, cuenta cuántas publicaciones tiene cada correo y las almacena en una lista adicional (`cabezaLista`). Luego, ordena esta lista usando el algoritmo de burbuja para que los correos con más publicaciones aparezcan primero. Finalmente, muestra los cinco correos con más publicaciones y limpia la memoria de la lista auxiliar.

El método `buscarPublicacionesPorEmail` permite buscar y agregar todas las publicaciones correspondientes a un correo electrónico específico en una lista circular doble (`listaCircular`), mientras que `buscarPublicacionesPorListaEmails` realiza una búsqueda similar pero para todos los correos en una lista de amigos (`listaEmails`).

Para representar gráficamente la lista de publicaciones, el método `generateDot` crea un archivo en formato DOT que describe la lista en términos de nodos y conexiones. El método `renderGraphviz` usa Graphviz para convertir este archivo DOT en una imagen PNG y luego la abre automáticamente, adaptando el comando para distintos sistemas operativos.

```

1 #include "listaPublicaciones.h"
2 #include "listaCircularDoble.h"
3 #include "listaEnlazadaDoble.h"
4 #include "nodoCorreo.h"
5 #include <iostream>
6
7 // Constructor y Destructor
8 listaDePublicaciones(listaDePublicaciones(usuario* listaDeUsuarios) : cabeza(nullptr), cola(nullptr), listaDeUsuarios(listaDeUsuarios) {}
9
10 listaDePublicaciones::~listaDePublicaciones() {}
11 while (cabeza != nullptr) {
12     eliminarPublicacion(cabeza->correo, cabeza->fecha, cabeza->hora);
13 }
14 }
15
16 // Métodos de agregar y eliminar publicaciones
17 void listaDePublicaciones::agregarPublicacion(const std::string& correo, const std::string& contenido, const std::string& fecha, const std::string& hora) {
18     if ((listaDeUsuarios->encontrar(correo)) {
19         std::cout << "El correo electrónico del usuario no está registrado." << std::endl;
20         return;
21     }
22     if (contenido.empty() || fecha.empty() || hora.empty()) {
23         std::cout << "Todos los campos deben estar completos." << std::endl;
24         return;
25     }
26     Publicacion* nuevaPublicacion = new Publicacion(correo, contenido, fecha, hora);
27     if (cola == nullptr) {
28         cabeza = cola = nuevaPublicacion;
29     } else {
30         cola->siguiente = nuevaPublicacion;
31         nuevaPublicacion->anterior = cola;
32         cola = nuevaPublicacion;
33     }
34 }

```

Lista Simple

La clase ListaSimple es una implementación para gestionar una lista de solicitudes de amistad. Se inicia con un constructor que configura el puntero cabeza en nullptr, lo que indica que la lista está vacía al principio. Al destruir el objeto, el destructor recorre toda la lista desde la cabeza y elimina cada nodo para evitar pérdidas de memoria.

Cuando se quiere agregar una nueva solicitud de amistad, se usa el método agregarSolicitud. Este método crea un nuevo objeto de tipo SolicitudAmistad con el emisor, receptor y estado proporcionados, y lo inserta al principio de la lista. Esto asegura que las solicitudes más recientes se encuentren al inicio.

Para eliminar una solicitud, se utiliza el método eliminarSolicitud. Este método busca una solicitud que coincida con el emisor, receptor y estado dados. Dado que las solicitudes pueden ser bidireccionales, el método verifica ambas direcciones. Si encuentra una coincidencia, ajusta los enlaces de los nodos adyacentes para remover la solicitud de la lista y luego elimina el nodo.

El método existeSolicitudPendiente se utiliza para verificar si existe alguna solicitud en la lista con el estado "PENDIENTE" entre el emisor y receptor especificados. Si encuentra una solicitud pendiente, devuelve true; si no, devuelve false. De manera similar, existeSolicitudAceptada verifica si hay una solicitud aceptada entre el emisor y receptor dados, devolviendo true si la encuentra y false si no es así.

El método imprimirLista recorre la lista e imprime la información de cada solicitud, mostrando quién envió la solicitud, a quién se envió y su estado actual.

Para buscar solicitudes en función del receptor o el emisor, están los métodos buscarPorReceptor y buscarPorEmisor. El primero busca todas las solicitudes en las que el receptor coincide con el receptor dado y agrega estas solicitudes a una pila, mientras que el segundo busca todas las solicitudes enviadas por el emisor especificado.

```

1 #include "ListaSimple.h"
2 #include <iostream>
3
4 ListaSimple::ListaSimple() : cabeza(nullptr) {}
5
6 ListaSimple::~ListaSimple() {
7     while (cabeza != nullptr) {
8         Solicitud* temp = cabeza;
9         cabeza = cabeza->siguiente;
10        delete temp;
11    }
12 }
13
14 void ListaSimple::agregarSolicitud(const std::string& emisor, const std::string& receptor, const std::string& estado) {
15     Solicitud* nuevaSolicitud = new Solicitud(emisor, receptor, estado);
16     nuevaSolicitud->siguiente = cabeza;
17     cabeza = nuevaSolicitud;
18 }
19
20 void ListaSimple::eliminarSolicitud(const std::string& emisor, const std::string& receptor, const std::string& estado) {
21     Solicitud* actual = cabeza;
22     Solicitud* previo = nullptr;
23
24     while (actual != nullptr) {
25         if (((actual->emisor == emisor && actual->receptor == receptor) ||
26             (actual->emisor == receptor && actual->receptor == emisor)) &&
27             actual->estado == estado) {
28             if (previo == nullptr) {
29                 cabeza = actual->siguiente;
30             } else {
31                 previo->siguiente = actual->siguiente;
32             }
33             delete actual;
34             actual = (previo != nullptr) ? previo->siguiente : cabeza;
35         } else {
36             previo = actual;
37             actual = actual->siguiente;
38         }
39     }
40 }
41
42 bool ListaSimple::existeSolicitudPendiente(const std::string& emisor, const std::string& receptor) const {
43     Solicitud* actual = cabeza;
44     while (actual != nullptr) {
45         if (((actual->emisor == emisor && actual->receptor == receptor) ||
46             (actual->emisor == receptor && actual->receptor == emisor)) &&
47             actual->estado == "PENDIENTE") {
48             return true;
49         }
50         actual = actual->siguiente;
51     }
52     return false;
53 }

```

Matriz

La clase Matrix gestiona una estructura de datos donde los nodos representan celdas en una matriz, y cada nodo tiene un valor booleano y nombres para las filas y columnas. La clase Node define los nodos que forman la matriz, cada uno con sus propios índices de fila (i), columna (j), valor (value), y nombres asociados para las filas y columnas (nombrei y nombrej).

El método insert agrega un nuevo nodo a la matriz. Primero, se crea un nuevo nodo con los índices, valor, y nombres proporcionados. Si la raíz de la matriz es nula, se inicializa con un nodo de cabecera. Luego, se buscan los nodos de fila y columna correspondientes. Si la columna o la fila no existen, se crean e insertan como cabeceras. Si el nodo que se está agregando ya existe, se actualiza su valor; de lo contrario, se inserta en la fila y la columna adecuadas.

El método searchRow recorre la lista de nodos en busca de una fila específica, mientras que searchColumn hace lo mismo para las columnas. Estos métodos permiten encontrar rápidamente si una fila o columna ya está presente en la matriz.

Para verificar si un nodo ya existe, el método nodeExists recorre las filas y columnas en busca de un nodo con el mismo índice de fila y columna, actualizando el valor si lo encuentra o indicando que el nodo no existe si no lo encuentra.

El método insertRowHeader crea e inserta un nodo de cabecera para una nueva fila, y insertColumnHeader hace lo mismo para las columnas. Los métodos insertInRow e insertInColumn insertan nodos en las filas y columnas, respectivamente, manteniendo el orden adecuado.

El método print imprime la matriz en la consola, mostrando los valores de las celdas con un formato de tabla. printColumnHeaders imprime los encabezados de columna para una visualización clara.

El método getValue devuelve el valor de un nodo en una posición dada, buscando a través de las filas y columnas.

Para generar una representación gráfica de la matriz, generateGraphvizImage utiliza el formato DOT para crear un archivo de gráficos, que luego se convierte en una imagen PNG usando Graphviz. El archivo temporal DOT se elimina después de la conversión.

```

1  #include "Matrix.h"
2  #include <fstream>
3  #include <cstdlib> // Para system()
4  #include <iomanip>
5  #include "ListaSimpleAmigos.h"
6
7  Node::Node(int i, int j, bool value, string nombrei, string nombrej)
8      : i(i), j(j), value(value), nombrei(nombrei), nombrej(nombrej) {}
9
10 void Matrix::Insert(int i, int j, bool value, const string& nombrei, const string& nombrej) {
11     Node* newNode = new Node(i, j, value, nombrei, nombrej);
12
13     if (!root) {
14         root = new Node(-1, -1);
15     }
16
17     Node* row = searchRow(i);
18     Node* column = searchColumn(j);
19
20     if (j > width) width = j;
21     if (i > height) height = i;
22
23     if (!nodeExists(newNode)) {
24         if (!column) {
25             column = insertColumnHeader(j);
26         }
27
28         if (!row) {
29             row = insertRowHeader(i);
30         }
31         insertInColumn(newNode, row);
32         insertInRow(newNode, column);
33     }
34 }

```

Pila de Solicitudes

La clase PilaSolicitudes tiene varios métodos para manejar esta pila. En el constructor PilaSolicitudes(), se inicializa el puntero tope como nullptr, indicando que la pila está vacía al principio. El destructor ~PilaSolicitudes() llama a limpiarPila(), que recorre y elimina todos los nodos de la pila para liberar la memoria al destruir el objeto.

El método pushSolicitud agrega una nueva solicitud a la pila solo si no existe una solicitud pendiente entre el emisor y el receptor en ambos sentidos. Se crea un nuevo nodo NodoSolicitud con los detalles de la solicitud y se inserta en la parte superior de la pila. Si ya existe una solicitud pendiente, se muestra un mensaje indicando que no se puede agregar.

El método popSolicitud elimina el nodo en la parte superior de la pila, si es que la pila no está vacía. Si la pila está vacía, se muestra un mensaje indicando que no se puede realizar la operación.

Para imprimir el contenido de la pila, el método imprimirPila recorre todos los nodos de la pila desde el tope y muestra el emisor de cada solicitud en la consola.

El método existeSolicitudPendiente verifica si existe una solicitud pendiente entre un receptor y un email dado. Recorre todos los nodos de la pila y compara los detalles de cada solicitud con los parámetros proporcionados. Si encuentra una solicitud con estado "PENDIENTE", retorna true; de lo contrario, retorna false.

La función limpiarPila es utilizada para eliminar todos los nodos de la pila, asegurando que toda la memoria utilizada por la pila sea liberada.

Para generar una representación gráfica de la pila, el método generarDot crea un archivo en formato DOT, que describe la estructura de la pila en un formato que puede ser interpretado por herramientas como Graphviz. Se especifican las propiedades visuales de los nodos y las conexiones entre ellos en el archivo DOT. Cada nodo en la pila se representa con un identificador único y contiene información sobre el emisor, receptor y estado de la solicitud.

El método renderGraphviz ejecuta el comando dot para convertir el archivo DOT en una imagen PNG, utilizando la herramienta Graphviz.

```

1 #include "PilaSolicitudes.h"
2 #include <iostream>
3 #include <cstdlib>
4
5 PilaSolicitudes::PilaSolicitudes() : tope(nullptr) {}
6
7 PilaSolicitudes::~PilaSolicitudes() {
8     limpiarPila();
9 }
10
11 void PilaSolicitudes::pushSolicitud(const std::string& emisor, const std::string& receptor, const std::string& estado) {
12     if (!existeSolicitudPendiente(emisor, receptor) && !existeSolicitudPendiente(receptor, emisor)) {
13         NodoSolicitud* nuevaSolicitud = new NodoSolicitud(emisor, receptor, estado);
14         nuevaSolicitud->siguiente = tope;
15         tope = nuevaSolicitud;
16     } else {
17         std::cout << "Ya existe una solicitud pendiente entre " << emisor << " y " << receptor << "." << std::endl;
18     }
19 }
20
21 void PilaSolicitudes::popSolicitud() {
22     if (tope != nullptr) {
23         NodoSolicitud* temp = tope;
24         tope = tope->siguiente;
25         delete temp;
26     } else {
27         std::cout << "La pila está vacía." << std::endl;
28     }
29 }
30
31 void PilaSolicitudes::imprimirPila() const {
32     NodoSolicitud* actual = tope;
33     while (actual != nullptr) {
34         std::cout << actual->emisor << endl;
35         actual = actual->siguiente;
36     }
37 }

```

Gestion Solicitudes de Amistad

La clase RedSocial tiene un constructor que toma un puntero a UserList, el cual es utilizado para verificar si los usuarios involucrados en las solicitudes están registrados. El método cargarSolicitudesDesdeJSON es responsable de leer solicitudes de amistad desde un archivo JSON y procesarlas para actualizar el estado de las solicitudes en la red social.

Al inicio del método cargarSolicitudesDesdeJSON, se abre el archivo especificado por filePath. Si el archivo no se puede abrir, se imprime un mensaje de error y se sale del método. Una vez abierto, el archivo se lee y se convierte en un objeto JSON utilizando la biblioteca nlohmann::json.

Cada solicitud en el JSON se procesa en un bucle. Se extraen los datos de emisor, receptor y estado de la solicitud. Antes de agregar la solicitud a la lista, se verifica que ambos correos electrónicos estén registrados en listaUsuarios. Si alguno de los correos electrónicos no está registrado, se muestra un mensaje de error y se omite la solicitud.

Luego, se verifica si ya existe una solicitud aceptada o pendiente entre el emisor y el receptor. Si el estado de la solicitud es "PENDIENTE", se revisa si ya existe una solicitud aceptada entre los mismos usuarios. Si existe una solicitud aceptada, se eliminan todas las solicitudes pendientes anteriores entre ellos, tanto del emisor al receptor como del receptor al emisor. Si no existe una solicitud pendiente, se agrega la nueva solicitud pendiente a la lista y se muestra un mensaje de confirmación.

Si el estado de la solicitud es "ACEPTADA", el método primero verifica si existe una solicitud pendiente entre el emisor y el receptor. Si existe, se elimina la solicitud pendiente antes de cambiar el estado a "ACEPTADA". Si no hay ninguna solicitud aceptada previa, se agrega la solicitud aceptada a la matriz de amigos y se actualizan las entradas correspondientes en ambas direcciones (es decir, tanto del emisor al receptor como del receptor al emisor). Se imprime un mensaje confirmando que la solicitud ha sido añadida con éxito como aceptada.

```
1 #include "RedSocial.h"
2 #include <fstream>
3 #include <iostream>
4 #include <json.hpp>
5
6 using json = nlohmann::json;
7 using namespace std;
8
9 // Implementación del constructor
10 RedSocial::RedSocial(UserList* listaUsuarios) : listaUsuarios(listaUsuarios) {}
11
12 // Implementación del método para cargar solicitudes desde un archivo JSON
13 void RedSocial::cargarSolicitudesDesdeJSON(const std::string& filePath) {
14     std::ifstream archivo(filePath);
15     if (!archivo.is_open()) {
16         cerr << "No se pudo abrir el archivo " << filePath << endl;
17         return;
18     }
19
20     json solicitudesJSON;
21     archivo >> solicitudesJSON;
22 }
```

Lista de Usuarios

La clase `UserList` comienza con un constructor que inicializa la lista de usuarios como vacía (primero es `nullptr`), establece el tamaño de la lista en 0 y asigna un identificador inicial de 1 a los usuarios. El destructor se encarga de liberar la memoria ocupada por los usuarios al recorrer la lista y eliminar cada nodo.

El método `emailExists` verifica si un correo electrónico específico ya está registrado en la lista. Recorre la lista de usuarios y compara el correo electrónico de cada usuario con el proporcionado. Si encuentra una coincidencia, retorna `true`, de lo contrario, al finalizar la lista, retorna `false`.

El método `addUser` permite agregar un nuevo usuario a la lista. Primero verifica que todos los campos necesarios (nombre, apellido, fecha de nacimiento, correo electrónico y contraseña) no estén vacíos. Luego, verifica si el correo electrónico ya está registrado. Si el correo electrónico es nuevo, crea un nuevo usuario con un identificador único, lo inserta al inicio de la lista y actualiza el tamaño de la lista. Imprime un mensaje confirmando la adición del usuario.

El método `printUsers` imprime los detalles de todos los usuarios en la lista. Recorre la lista y muestra el identificador, nombre completo, correo electrónico y contraseña de cada usuario. La fecha de nacimiento está comentada en el código.

El método `authenticateUser` verifica las credenciales de un usuario. Recorre la lista de usuarios y compara el correo electrónico y la contraseña proporcionados con los almacenados en cada nodo. Si encuentra una coincidencia, retorna `true`, de lo contrario, retorna `false`.

El método `deleteUserByEmail` elimina un usuario de la lista basado en su correo electrónico. Recorre la lista, buscando el usuario con el correo electrónico especificado. Si lo encuentra, ajusta los punteros de la lista para excluir el nodo del usuario a eliminar y libera la memoria asociada. Si no encuentra el usuario, imprime un mensaje indicando que el usuario no fue encontrado.

El método `getSize` retorna el número de usuarios en la lista.

Los métodos `foundUser` y `getBirthDateByEmail` buscan un usuario por su correo electrónico y retornan el nombre completo y la fecha de nacimiento del usuario, respectivamente. Si el correo electrónico no está en la lista, retornan una cadena vacía.

El método `getId` devuelve el identificador de un usuario basado en su correo electrónico. Si el usuario no se encuentra, retorna -1.

Finalmente, el método `generateDot` crea un archivo DOT que describe la lista de usuarios en un formato gráfico. Este archivo puede ser usado por Graphviz para generar una representación visual de la lista. El método `renderGraphviz` ejecuta un comando del sistema para convertir el archivo DOT en una imagen PNG usando Graphviz.

```
1  #include "UserList.h"
2  #include <iostream>
3  #include <cstdlib>
4
5  UserList::UserList() : primero(nullptr), tamaño(0), usrId(1) {}
6
7  UserList::~UserList() {
8      User* actual = primero;
9      while (actual != nullptr) {
10         User* limpiable = actual;
11         actual = actual->siguiente;
12         delete limpiable;
13     }
14 }
15
16 bool UserList::emailExists(const string& atrEmail) const {
17     if (atrEmail.empty()) {
18         return false;
19     }
20
21     User* actual = primero;
22     while (actual != nullptr) {
23         if (actual->atrEmail == atrEmail) {
24             return true;
25         }
26         actual = actual->siguiente;
27     }
28     return false;
29 }
```